Python Engineering

Class 05 - Lists & Tuples

References, mutability, and why trajectory appeared to change by itself

Support note for **week_02_data_structures -> class_05_lists_tuples**. This document explains the behavior observed when storing lists inside a tuple in `trajectory`.

1. The observed behavior

The issue appeared after creating three lists to store the time history of altitude, velocity, and time, and then storing those lists inside trajectory.

```
h = []
v = []
t = []

h.append(h0)
v.append(v0)
t.append(t0)

trajectory.append((h, v, t))
```

That final append does not store the initial scalar values. It stores a tuple containing **references to the three list objects**.

2. Reference, not snapshot

```
h ----------------------> [0]
                           ^
trajectory[0][0] ----------|

v ----------------------> [50]
trajectory[0][1] ----------|

t ----------------------> [0]
trajectory[0][2] ----------|
```

When you later execute `h.append(h_new)`, Python does not create a new list. It modifies the same object already referenced by trajectory.

3. Why the first element kept growing

```
h.append(5.0)
v.append(49.019)
t.append(0.1)
```

The lists now contain more values, and because trajectory still references those same objects, printing trajectory shows something like:

```
[
  ([0, 5.0, 9.90, ...],
   [50, 49.019, 48.038, ...],
   [0, 0.1, 0.2, ...]),

  (5.0, 49.019, 0.1),
  (9.90, 48.038, 0.2),
  ...
]
```

4. But aren't tuples immutable?

Yes. A tuple is immutable with respect to its own structure: you cannot freely replace its elements. However, a tuple may contain mutable objects such as lists, and those objects can still change.

```
my_list = [1]
my_tuple = (my_list,)

my_list.append(2)

print(my_tuple)
# ([1, 2],)
```

Key idea: an immutable tuple does not imply deeply immutable contents.

5. The technical name: aliasing

When two names or data structures point to the same mutable object, we have **aliasing**. Changing the object through one reference makes that change visible through the other references.

6. What you actually wanted to store

Each element of trajectory should represent one **complete scalar state** of the simulation, not three entire histories.

```
trajectory.append((t0, h0, v0))
```

and inside the loop:

```
trajectory.append((t_new, h_new, v_new))
```

Then trajectory becomes a list of states:

```
trajectory = [
    (0.0, 0.0, 50.0),
    (0.1, 5.0, 49.019),
    (0.2, 9.9019, 48.038),
    ...
]
```

7. Two valid representations of the same trajectory

Three lists by variable	List of tuples by state
times = [...] heights = [...] velocities = [...]	trajectory = [(t0,h0,v0), (t1,h1,v1), ...]
Very convenient for plotting or operating by variable.	Very convenient for working with complete states by epoch.

8. What if you really wanted a copy of the lists?

If you truly wanted to freeze the current list contents at a given instant, you could make copies:

```
trajectory.append((h.copy(), v.copy(), t.copy()))
```

But this is not the desired EX13 structure, because each element should represent a state `(t, h, v)`, not a complete history.

9. Recommended EX13 structure

```
h = []
v = []
t = []
trajectory = []

h.append(h0)
v.append(v0)
t.append(t0)
trajectory.append((t0, h0, v0))

while h[i] >= 0:
```

```
    h_new, v_new, t_new = euler_step(h[i], v[i], t[i], g, dt)

    h.append(h_new)
    v.append(v_new)
    t.append(t_new)
    trajectory.append((t_new, h_new, v_new))

    i += 1
```

10. Learning Tracker

Concept	Status
List as a mutable object	Learned
Tuple as an immutable structure	Learned
Mutable object inside a tuple	Learned
Reference vs copy	Learned
Aliasing	Introduced
Variable histories vs state history	Learned
copy()	Introduced

Remember: Python does not normally store a snapshot of a list when you place it inside another structure; it stores a reference to the same object. If that mutable object changes, the change is visible through all references to it.